Assignment 1 剩余题详解

Stanford CS336 • Spring 2026 从已完成的TinyStories实验出发

完整实验分析、逐步资源核算、架构消融预测与代码待办。采用竖版单栏、嵌入中文字体和章节书签，适合手机逐页阅读。

真实结果：160000步，验证loss 1.33946。 实验配置：22.70M参数，327.68M训练token。

重要标记：实测是已有证据；理论是公式推导；预测不充当实验记录。本书没有额外启动训练。

包含两张由真实日志绘制的学习曲线。 整理日期：2026年10月28日。

目录

01 阅读导航与证据边界

02 已完成的基础实验

03 experiment_log：怎样记录实验

04 learning_rate：扫描与公平比较

05 learning_rate：稳定性边缘

06 batch_size_experiment：固定预算

07 batch：资源与实验解释

08 decoding：从分数到下一个token

09 generate：如何解释输出质量

10 layer_norm_ablation：去掉RMSNorm

11 pre_norm_ablation：位置为何重要

12 no_pos_emb：RoPE与NoPE

13 swiglu_ablation：门控与参数匹配

14 main_experiment：OpenWebText

目录（续）

15 leaderboard：一个可解释的修改

16 Transformer资源核算：参数

17 Transformer资源核算：FLOPs

18 资源核算：规模与长上下文

19 AdamW核算：参数与激活

20 AdamW核算：80 GB与更新成本

21 AdamW核算：MFU与训练时间

22 BPE遗留题：训练与瓶颈

23 Tokenizer遗留题：压缩率与吞吐

24 SGD学习率小题：可以直接推导

25 Unicode补充题

26 代码待办：范围与验收标准

27 交付清单与完成边界

28 来源与进度记录

阅读导航与证据边界

本讲义把上一轮 Assignment 1 剩余题答案扩写为可以逐页阅读的学习笔记。依据本地中文讲义26.0.3及已核对的作业记录，整理于2026年10月28日。

【实测】仅用于已经收到或读到的日志、文件和检查结果。【理论】表示由公式或代码定义推导得到。【预测】表示未运行实验的合理预期，不代表模型实际表现。

正式实验的真实结果为：峰值学习率0.001，batch 8，训练160000步，最终采样验证loss为1.33945854。其他学习率、batch和消融实验没有对应的实测曲线。

本次不启动训练，也不修改D盘作业代码。需要补充的实现集中列在最后。真实学习曲线直接来自已有JSONL，预测部分不绘制伪造曲线。

阅读顺序：先读实验与生成，再读架构消融；资源核算可单独阅读。PDF侧栏包含章节书签，下一页提供可点击目录。

概念检查：理论预测能替代实验曲线吗？答案：不能。它能解释机制，但不能提供实际最终loss、发散步数或生成文本。

已完成的基础实验

模型结构：词表10000，上下文256，宽度512，4层、16头，SwiGLU内部维度1344，RoPE theta为10000。Embedding与LM head未共享权重，共22,696,448个参数。

优化配置：AdamW，beta1=0.9、beta2=0.999、eps=1e-8、weight decay=0.01；全局梯度裁剪阈值1.0。峰值学习率0.001，最小0.0001，warmup 1600步，cosine终点160000。

每步处理8×256=2048个token，总计327,680,000个token。这是累计处理量，随机采样可能重复，不等于顺序读取同样数量的独立token。

最终验证loss为1.33945854，是记录中的最低值；首次记录到loss≤1.45是在84000步，为1.44974236。每1000步才验证一次，因此不能确定精确跨过阈值的更新步。

checkpoint已读回，iteration为160000；模型39个状态张量全部有限，优化器有39项状态和1个参数组。这不等于所有官方单元测试均已确认通过。

验证采用固定32个batch，即65536个采样token，而不是完整验证集。运行计时约5小时18秒，不含数据扫描和模型创建。

experiment_log：怎样记录实验

小例子：第100步loss为3.0、第200步为2.5；如果两点对应10秒和50秒，那么“每步进展”和“单位时间进展”描述的是不同问题。官方要求同时按更新步数和wall-clock time展示曲线。

可写入报告：本实验用config.json保存配置，用metrics.jsonl记录step、learning rate、train_loss、valid_loss及elapsed_seconds，用last.pt保存模型、优化器状态与已完成步数。

train_loss是当前单个batch的平均损失，不是日志区间平均。valid_loss为固定采样batch的平均值。比较两者时，不能把某一步训练loss较低直接判定为过拟合。

同一实验的last.pt会被后续保存覆盖。checkpoint没有保存随机数状态，因此恢复训练不能保证与不中断运行逐步一致。恢复后的elapsed_seconds从新进程重新计时；拼图时应按运行分段处理。

需要的新代码：通用日志绘图脚本。应读取真实JSONL，分别画步数与时间横轴，注明是否平滑、是否跨恢复段。本PDF已经为当前这一个连续正式运行绘图，不代表通用脚本已接入作业仓库。

检查：看到训练loss的尖峰是否就是发散？答案：不是，单batch差异也会造成尖峰，要结合持续趋势和验证结果。

learning_rate：扫描与公平比较

小例子：若更新方向固定，学习率由0.001升到0.003，当前步更新幅度约变为3倍。但下一步的参数、梯度、AdamW状态也会改变，所以训练速度不会机械地变成3倍。

【实测】0.001完成160000步，最终采样验证loss为1.33945854；这说明它是一个有效配置，但不能证明它是最优学习率。0.003没有运行，最终loss必须留为“未知”。

搜索策略答案：先按倍数粗搜，例如0.001、0.003及更大候选；找到表现较好或开始不稳定的区间后再细搜。候选值是搜索设计，不是已经完成的实验。

公平比较应固定模型、初始化seed、数据采样设置、batch、token预算和warmup比例。若最小学习率保持峰值的10%，比较的是整个调度的幅度；报告应明确这一点。

【预测】0.003可能更快降低早期loss，也可能出现更大波动或更差最终结果。已有0.001曲线不足以支持具体损失数值。

新代码：现有训练CLI已支持这些配置，不必新增优化器。尚缺多学习率实际曲线；按你的要求不追加训练，保留为未完成实测项。

learning_rate：稳定性边缘

在一维二次函数L(w)=a×w²/2上，普通梯度下降给出w_next=(1-lr×a)×w。若乘数绝对值小于1，幅度缩小；若大于1，幅度放大。

例如a=1，lr=0.5时每步变为原来一半；lr=2.5时每步乘以-1.5，符号来回变化且幅度增大。这解释了过大学习率为什么可能导致震荡和发散。

这只是解释机制的玩具模型。真实Transformer使用AdamW、warmup和梯度裁剪，不能把lr<2这样的界限直接移用到语言模型。

可写入报告：较大学习率可以加快优化，但过大可能使更新反复越过低损失区域。最佳学习率可能靠近稳定范围上界，但并非定律；其位置取决于架构、batch、初始化、优化器和调度。

一次loss上升不是发散。较有说服力的迹象包括长期持续恶化、巨大振荡或非有限loss/梯度。OOM是资源不足，不是学习率发散。

官方要求至少一条发散运行曲线。当前没有该证据，所以只能讨论预测，不能写“已找到稳定性边缘”。

batch_size_experiment：固定预算

先用小数字理解：每条序列8个token，batch为2，每步处理16个token；预算160个token需要10步。batch改成4，同样预算只需要5步。

通用关系：token预算 = batch size × steps × context length。本次预算327,680,000，context为256。

对应步数：batch1需要1,280,000步；batch8需要160,000步；batch32需要40,000步；batch64需要20,000步；batch128需要10,000步。这是数学换算，不是显存适配结果。

【预测】大batch通常减少梯度估计噪声，在一定范围内提高GPU吞吐量，但增加激活显存。固定token预算时，大batch的参数更新次数更少，最终loss不一定更低，学习率也可能需要重新调优。

更平滑的loss可能只是平均样本更多，不等于泛化更好。更高tokens/s也不自动等于更快达到同一验证loss。应同时看统计效率和硬件效率。

【实测边界】当前确认batch8可持续训练；batch1只完成过2步设备检查。不能把这两种记录当成公平的batch对照实验。

batch：资源与实验解释

讲义要求尝试从1到显存允许的范围，并包含64、128等典型值；但不能声称6 GiB显卡在当前float32实现下可容纳这些物理batch。实际不支持时应记录限制。

梯度累积可以把多个microbatch的梯度组合后再更新。例如4次batch8可以形成有效batch32，但需要正确缩放loss，并且4次反向后只执行一次optimizer.step。

累积可以近似较大batch的梯度，却不会获得同样的大矩阵并行效率。因此“有效batch32”和“物理batch32”的吞吐与显存不能混为一谈。

若改变batch，为了比较相同训练阶段，warmup和cosine长度也应结合步数预算重新确定，或明确使用按token定义的调度。不能机械复制160000步，否则总token预算改变。

可写入报告：预计存在硬件吞吐、优化噪声和更新次数之间的权衡；最佳batch需结合学习率调优确定。当前只有batch8的完整学习曲线，尚不能作跨batch实测结论。

需要代码：基础CLI已有batch选项；若要测峰值显存需增加测量，若要梯度累积则需修改训练循环。本次均不执行。

decoding：从分数到下一个token

假设词表3项，最后位置logits为[2,1,0]。softmax后约为[0.665,0.245,0.090]。logits只是分数，概率则非负且和为1。

输入ID张量形状是(1,T)，模型输出为(1,T,10000)。每个位置预测该位置之后的token，因此生成只取最后一个位置，得到形状(10000,)的候选分数。

温度缩放：p_i = exp(v_i/τ) / sum_j exp(v_j/τ)。τ小于1通常使分布更集中，τ大于1使其更平坦。脚本额外约定τ=0为greedy decoding，直接取最大分数。

top-p例子：概率[0.60,0.25,0.10,0.05]，p=0.8，保留前两项，因为0.60不够、0.85首次达到阈值。重新归一化后约为[0.706,0.294]。

注意保留“使累计概率首次达到阈值”的那一项，不能简单删除所有累计概率大于p的位置。排序后的索引也必须映射回原始词表ID。

自回归过程：每次生成一个token并追加到历史；遇到结束token或达到最大新增token数停止。代码已保存，但运行成功尚未确认。

generate：如何解释输出质量

正式题目要求至少256个token的输出，或在第一个结束token处停止。此前演示的100，以及脚本默认200，可能在未结束时过早截断。正式采样应将max-new-tokens至少设为256；无需因此改写生成算法。

【预测】训练后的模型可能生成常见英文句式、简单角色和短情节，但仍可能有重复、代词混乱或情节跳跃。没有实际输出，不能断言某段文本已经流畅，也不能编写故事冒充模型样本。

影响质量的因素至少包括：训练语料及模型容量；temperature与top-p；上下文长度。低温通常更确定但可能重复，高温可能更多样却不连贯；过小top-p也可能让输出单调。

当前脚本只把最近256个token送入模型，完整历史用于显示。滑动窗口能避免RoPE缓存越界，但更早的细节不会直接参与预测，长故事可能忘记人物或设定。

输入新开头会开始独立故事，/continue才使用上一段历史。这个交互界面不会把TinyStories预训练模型自动变成指令问答模型。

代码需求：核心生成函数已有；保存prompt、seed、采样参数、输出文本和停止原因是可补充的记录功能。正式报告中的输出仍应来自真实运行。

layer_norm_ablation：去掉RMSNorm

小例子：向量[3,4]的均方根约为3.536，除以它后约为[0.849,1.131]，均方根约为1。实际还包含epsilon与可学习缩放参数，且RMSNorm不减去均值。

RMSNorm帮助控制送入子层的激活尺度。在pre-norm结构中，残差流可持续积累，但进入attention和FFN的表示先经过归一化，降低了子层对整体幅度变化的敏感性。

【预测答案】移除RMSNorm后，激活与梯度尺度可能更难控制，在原学习率0.001下更容易震荡或不稳定。降低学习率可能恢复部分稳定性，但不保证恢复相同的收敛速度或最终loss。

不能写成“必然发散”：模型只有4层，初始化、AdamW和裁剪可能让它仍能训练。判断必须来自曲线，而非仅凭归一化的通常作用。

题目要求移除所有RMSNorm，应包含各块ln1、ln2及模型末尾ln_final。只删块内两处却保留最终层，属于不同实验，必须说明。

代码需求：增加关闭归一化的架构选项，并写入配置。消融应从头训练；直接拆掉已训练模型中的Norm后评估，测的是另一种干预。

pre_norm_ablation：位置为何重要

Pre-norm先归一化，再进入子层，最后残差相加：z=x+Attn(Norm(x))；y=z+FFN(Norm(z))。

Post-norm先让子层处理输入并做残差相加，再归一化：z=Norm(x+Attn(x))；y=Norm(z+FFN(z))。

用简化表达理解梯度：pre-norm块可写作y=x+F(Norm(x))，其导数包含一个直接的恒等项I。post-norm的整个残差和都经过Norm，导数还要乘归一化的Jacobian。

这并不表示pre-norm完全没有梯度问题，只说明它保留了一条较直接的残差传播路径。深度增加时，这种差别通常更值得关注。

【预测答案】Post-norm可能对初始化、学习率和warmup更敏感，需要更谨慎调参。对于当前4层模型，差距也可能较小，不能预先断言必然失败或最终loss一定更高。

代码需求：TransformerBlock增加norm位置选项，训练和生成共享同一结构配置。最终ln_final保持基础设置，避免同时改动另一个因素。这里不需要改写attention或FFN本身。

no_pos_emb：RoPE与NoPE

Attention中的点积本身比较Q和K的内容。RoPE让不同位置使用不同旋转角，使旋转后的点积与相对位置有关。它主要作用在Q、K上，V不旋转。

例如“dog bites man”和“man bites dog”词相同，但顺序决定意思。显式位置机制为模型区分顺序与距离提供了结构上的帮助。

【预测答案】移除RoPE可能使词序、距离和某些依赖更难学习，导致收敛变慢或验证loss上升；但NoPE并非完全无位置信息。causal mask使各位置可见的前缀不同，模型仍可能从这种结构中获得位置线索。

因此不能把NoPE称为词袋模型，也不能保证在短而简单的TinyStories上会出现巨大性能下降。结果还可能随上下文长度变化。

代码需求：MultiHeadSelfAttention已有use_rope参数，可将该开关向TransformerBlock、TransformerLM和CLI传递。NoPE仍必须保留causal mask，不能把移除mask当成位置消融。

比较应使用相同训练预算、数据和初始化策略，分别从头训练。当前只有RoPE基础模型实测结果，NoPE曲线不存在。

swiglu_ablation：门控与参数匹配

SwiGLU形式：W2(SiLU(W1x) × W3x)。乘号表示对应元素相乘。SiLU分支不是限制在0到1的概率门，它能取负值，也能随输入改变幅度。

直观例子：两个分支对应元素为[0.2,2.0]和[5,1]，相乘得到[1,2]。不同通道可被不同程度地调节，再通过W2映射回模型维度。

无门控SiLU FFN为W2(SiLU(W1x))。若删掉一个矩阵却保持相同内部宽度，参数量减少，就同时改变了门控结构与容量，不能公平归因。

本题参数匹配：SwiGLU内部1344，每层参数
3×512×1344=2,064,384；SiLU内部2048，每层参数
2×512×2048=2,097,152，相差约1.59%。

【预测答案】SwiGLU可能通过输入相关调节提供更灵活的特征表达，在相近参数量下取得更好loss，但优势大小和方向仍需实验。不能报告一个未测得的固定差值。

代码需求：新增普通SiLU FFN及FFN类型选项，确认中间输出形状为(B,T,2048)，最终仍为(B,T,512)，并记录对应d_ff。

main_experiment：OpenWebText

TinyStories的词汇和情节相对简单。OpenWebText包含更广泛的主题、专名、数字、技术内容与不同写作风格，模型需要描述更复杂的分布。

【预测答案】相同容量和计算预算下，OWT模型可能只能学到局部语言规律，长段生成更容易主题漂移、事实不一致或缺乏结构。即使句子看起来像英语，也不等于事实正确。

平均交叉熵越低，表示模型给真实下一token分配的几何平均概率越高。困惑度PPL=exp(loss)，但其单位依赖tokenization。

例子：一个词被分成1个token与3个token，两种平均每token损失不能直接当作同一种难度尺度。比较10K TinyStories tokenizer和32K OWT tokenizer时尤其如此。

更可比的补充指标是总负对数似然除以原始字节数；若再除以ln(2)，得到bits per byte。仍要保证评价文本和预处理口径一致。

OWT保持暂缓。讲义也对GPU资源有限的在线学习者建议继续在TinyStories上测试修改，但这不自动变成已完成OWT题的证明。需要32K BPE、编码数据、曲线和真实生成文本才能给实测答案。

leaderboard：一个可解释的修改

Weight tying把输入embedding与输出LM head使用的权重绑定。当前两者形状均为(10000,512)，因此可以在结构上共享同一个参数，而不仅是复制一份数值。

参数减少10000×512=5,120,000，总参数从22,696,448变为17,576,448。float32训练若计参数、梯度和两个AdamW状态，理论减少约81,920,000字节，不含临时张量。

共享意味着输入和输出两个用途产生的梯度都累积到同一参数上。它减少自由度，也提供一种结构约束；不应把“参数更少”直接等同于“效果更好”。

【预测答案】可能节省存储并改善某些设置下的泛化，但也可能限制表达能力。输出词表投影仍然需要执行，所以训练FLOPs不会按参数减少比例同步下降。

需要模型代码与初始化配置修改。应新建实验配置，不修改既有checkpoint的含义。共享后最优初始化尺度和学习率可能变化。

讲义排行榜条件是单张B200最多45分钟，并只使用指定OWT训练数据。RTX3060的运行时间不能直接换算成符合条件的成绩。本地课程版本中的参考目标为loss优于5.0，并非我们已取得的结果。

Transformer资源核算：参数

统一符号：V是词表大小，T是上下文长度，L是层数，d是模型宽度，f是FFN内部宽度，h是头数，B是batch size。本节按无bias、输入输出权重不共享计算。

矩阵例子：(3,2)权重包含6个参数。参数量由权重形状决定，与一次输入多少token无关。

Embedding与LM head各有Vd个参数。每个块的Q、K、V和输出投影合计$4d^2$；SwiGLU三个矩阵合计3df；两个RMSNorm各有d个缩放参数。最终RMSNorm再增加d。

总参数：$P = 2Vd + L(4d^2 + 3df + 2d) + d$。

GPT-2 XL题设：V=50257，T=1024，L=48，d=1600，h=25，f=4288。代入P=1,640,452,800。

每个float32参数4字节，因此参数存储为6,561,811,200字节，约6.562 GB或6.111 GiB。这里GB按10的9次方，GiB按2的30次方；尚未包括梯度、激活和优化器。

检查：上下文变长会增加参数量吗？本作业RoPE缓存不是可训练参数，因此保持其他配置不变时，上述P不变，计算与激活开销会增加。

Transformer资源核算：FLOPs

按一次乘法加一次加法共2 FLOPs，矩阵乘法(m,k)×(k,n)的主导运算量为2mkn。例如(2,3)×(3,4)约需48 FLOPs。

对batch1、长度T的一层：QKV三个投影合计6Td²；attention输出投影2Td²；QK转置乘法2T²d；概率乘V也是2T²d；SwiGLU三个线性层6Tdf。

头数没有单独出现在两次attention乘法总量中，因为h×d_head=d。它仍会影响attention矩阵的存储量。

模型末端LM head再加2TdV。前向主导矩阵FLOPs：F = L(8Td² + 4T²d + 6Tdf) + 2TdV。

XL得到F=3,516,769,894,400，约3.517 TFLOPs/序列前向。batch为B时主导量约乘B。

这是稠密矩阵乘法口径，未计Norm、softmax、SiLU等逐元素运算。causal mask不会自动使普通稠密乘法少算一半；只有特定实现跳过运算时才应调整口径。

XL在T=1024时，FFN占57.53%，attention投影28.62%，两次attention乘法9.16%，LM head4.68%，所以主要开销来自FFN。

资源核算：规模与长上下文

保持V=50257、T=1024，f取最接近(8/3)d的64倍数。以下都是本作业架构，不是原版GPT-2的参数统计。

Small：L=12，d=768，f=2048。参数162,148,608，前向0.292 TFLOPs。投影/attention两次乘法/FFN/LM head占比为19.88% / 13.25% / 39.76% / 27.10%。

Medium：L=24，d=1024，f=2752。参数406,539,264，前向0.830 TFLOPs。四项占比24.83% / 12.42% / 50.05% / 12.70%。

Large：L=36，d=1280，f=3392。参数833,591,040，前向1.769 TFLOPs。四项占比27.32% / 10.93% / 54.30% / 7.45%。

模型增宽时，块内线性层近似按d^2增长；attention二次项仅按d线性增长。增深也使仅执行一次的LM head相对占比下降。

XL上下文增至16384，即16倍：线性于T的部分变16倍，attention二次项变256倍。总前向约133.578 TFLOPs，是原来37.98倍；attention两次乘法占61.73%，成为最大项。

AdamW核算：参数与激活

参数P个，每个float32占4字节。训练常驻项：参数4P、梯度4P、一阶矩4P、二阶矩4P，共16P字节。不包含optimizer临时张量。

激活不能仅由“使用AdamW”唯一决定：autograd保存策略、融合算子、重计算和张量生命周期都会影响真实峰值。下面采用题目列出算子各保留一份输出的教学估算。

每层激活元素数：$B[8Td + 4Tf + 2hT^2]$。8Td来自两次RMSNorm、QKV、attention加权结果、输出投影及FFN输出；4Tf来自W1、W3、SiLU及门控乘积；$2hT^2$来自分数与softmax概率。

块外再计最终RMSNorm的BTd、输出logits的BTV和交叉熵相关概率的BTV。因此$A = B\{L[8Td + 4Tf + 2hT^2] + Td + 2TV\}$。

A是元素数，乘4才是字节。总内存估算$M \approx 16P+4A$。

这不是精确实测峰值：残差、额外保存的输入、反向临时值、softmax实现及allocator均可能改变结果。报告应同时给出假设，不能只写一个“精确上限”。

AdamW核算：80 GB与更新成本

本题要求f=(8/3)d，因此本页使用理想未取整宽度，而不是Transformer核算中的4288。它用于代数估算，不是可直接创建的整数网络配置。

代入XL的其他数值，P=1,635,537,600。参数、梯度和两个状态共26,168,601,600字节。

按前页激活口径，每增加一个batch样本增加16,356,614,144字节。故M(B)≈16.356614144B + 26.168601600 GB。

80 GB下B≤(80-26.1686)/16.3566，取整数得到3。B=3约75.24 GB，B=4约91.60 GB。这个数只是该估算模型的容量，不是实际框架可用batch保证。

AdamW每参数更新的常见粗计数：一阶矩3次、二阶矩4次、归一化参数更新5次、权重衰减2次，共约14P FLOPs。平方根和除法各按1次，参数组级标量工作忽略。

主要思想：优化器成本线性于参数量P，而训练前反向还乘以batch和序列长度。对大batch长序列，矩阵乘法通常远比更新本身昂贵。

AdamW核算：MFU与训练时间

MFU必须无量纲。正确理解是：实际tokens/s乘以每token模型FLOPs，再除以硬件峰值FLOPs/s；不能直接把tokens/s除以FLOPs/s。

本题使用H100题设峰值495 TFLOP/s，并假设50% MFU，得到有效247.5 TFLOP/s。该495数字对应题目采用的TF32口径，不是普通标量FP32峰值。

用实际f=4288的XL前向F≈3.51677×10的12次方。若反向为前向2倍，每步训练主导FLOPs≈3BF；加上AdamW约14P。

batch1024、400000步：总时间 = 400000×(3×1024×F+14P) / (0.5×495×10的12次方)。

结果约4850小时，即202天。这是按固定吞吐率推算的理想化计算时间，未额外增加I/O、评估和故障开销。

batch1024与前页80 GB容量并不相容，实际需梯度累积或其他策略，而且不能保证仍达到假设MFU。本题是资源核算，不是建议你执行的训练计划。

检查：把MFU从50%提升到100%，其他条件不变会怎样？答案：理论时间减半；这不代表现实可以达到100%。

BPE遗留题：训练与瓶颈

【实测】TinyStories 10K BPE耗时508.08秒，得到10000个词表项、9743条合并规则；最长token为字节串“ accomplishment”，共15字节，含前导空格。

为什么合理：BPE反复合并高频相邻片段，常出现的完整英文单词连同前导空格可能成为一个token。最长token不必是最常见词，只是合并历史中的一个结果。

Python会话峰值物理工作集曾观察到约9.70 GiB，但它不是严格隔离训练阶段的峰值。时间低于30分钟；内存只能按观察口径报告，不能补成未经测量的精确峰值。

瓶颈题目前没有阶段profiling。可预测regex预分词、对象分配、pair计数和合并更新都可能耗时，但不能认定某一步占多少百分比。

重要事实：训练函数仍整文件读取；定义find_chunk_boundaries不等于已经调用或并行。正式token编码的分块策略也不能倒推成BPE训练已分块。

OWT 32K BPE仍暂缓。预测词表会更丰富，包含更多领域词片段、数字和标点组合；最长token、耗时和内存必须等训练后才能知道。跨语料比较也混合了10K与32K词表大小的影响。

Tokenizer遗留题：压缩率与吞吐

bytes/token = 原始UTF-8字节数 / 编码token数。例子：120字节编码为30个token，压缩率为4 bytes/token；数值越大，平均一个token覆盖更多原始字节。

完整TinyStories训练集：2,227,753,162字节 / 541,229,347 token ≈4.1161 bytes/token。这是全训练集汇总，不是题目要求的10篇抽样文档结果。

用TinyStories tokenizer编码OWT，字节级基础词表仍能表示文本，但领域词可能切得更碎。预测bytes/token可能较低，不能保证所有文档都如此。

端到端吞吐：2,227,753,162 / 5418.73 ≈411,121 bytes/s。总耗时包含检查、文件构建和读回，不是隔离的encode吞吐。

若825 GB按十进制且吞吐不变，外推约23.23天。Pile的文本分布、存储与处理策略不同，实际时间可能改变；这只能作为粗估。

uint16范围0到65535，足以存10K或32K词表ID，每个ID2字节。送入模型前再转torch.long。TinyStories训练/验证编码均已完成；OWT编码未完成。

SGD学习率小题：可以直接推导

题目有100个权重，loss是它们平方的均值：L=sum(w_i²)/100。因此每个分量的梯度是w_i/50。

题中SGD使用lr/sqrt(t+1)，第t步从0计数。更新为w_next=[1-lr/(50sqrt(t+1))]×w。loss会乘以上述因子的平方。

lr=10：首步因子0.8，随后保持正且小于1，loss逐渐减小。完成10次更新后的理论loss约为初始的0.11793倍。

lr=100：第一步因子-1，权重变号但loss不变；第二步因子约-0.414，幅度迅速减小；第四次更新，即t=3，理想算术下因子为0。

lr=1000：前10次更新的因子绝对值都大于1，符号反复改变且幅度增大，loss急剧放大。浮点运行可能与理想算术略有不同。

打印位置也重要：示例先打印loss再执行更新，所以第10次打印显示的是完成9次更新后的状态。不要将“10次打印”直接解释为“第10次更新后loss”。

可交付结论：10稳定下降，100先保持再快速下降，1000在这10次迭代内明显增大。这里是解析推导，没有伪造实测日志。

Unicode补充题

Unicode码点是字符的编号；UTF-8是把字符编号表示为字节序列的一种编码。字符、字节和BPE token是三个不同层次。

unicode1：chr(0)返回U+0000，即NUL。repr通常显示为“\x00”，打印时通常没有可见字形；具体终端表现可能不同。它仍占一个Python字符，不会自动截断Python字符串。

unicode2(a)：对于本作业大量英文及ASCII内容，UTF-8通常每个字符1字节，而UTF-16或UTF-32通常需要更多字节。UTF-8也能覆盖Unicode字符，但不保证对所有语言都是字节最少的编码。

unicode2(b)：é的UTF-8是两个字节0xC3、0xA9。逐字节分别decode会把完整多字节字符拆开，导致解码错误；应先拼接完整字节流再解码。

unicode2(c)：0xC3、0x28组成的双字节序列不是合法UTF-8，因为0xC3需要后续续字节，而0x28不是续字节。

这解释了生成脚本为什么最终对完整token字节序列解码，而不是把每个token单独转成字符串再拼接。单个token可能只包含多字节字符的一部分。

代码待办：范围与验收标准

绘图脚本：输入metrics.jsonl，输出步数与时间两套曲线；保留原始值，若平滑则标明方法。恢复运行必须分段处理计时，不能简单把时间列连成连续横轴。

架构开关：norm类型或位置、use_rope、FFN类型、可选weight tying。默认值应保持已通过测试的基础行为。新选项写入config，加载checkpoint时用相同配置重建模型。

SiLU FFN：新增两层无bias线性映射，中间SiLU，内部宽度2048；检查输入输出形状不变。不要为删除一个门控分支而大规模重写整个Transformer。

生成：已有完整generation.py，需真实运行确认。可补保存prompt、输出token IDs、解码参数、seed、checkpoint步数与停止原因。当前没有证据证明/continue等路径已经运行成功。

性能测量：若以后需要，加入峰值显存和同步后的计时。异步CUDA操作的计时必须明确同步点；不使用两步启动日志估算整场实验速度。

代码注释应解释“为什么”：维度含义、阈值边界、设备选择、checkpoint兼容与实验公平性，而不只是把语句翻译成中文。官方测试未确认项仍保持待确认，不改断言绕过测试。

交付清单与完成边界

已经有证据：BPE核心测试、Tokenizer大部分测试反馈、多个模型组件测试反馈、数据完整编码、CPU保存恢复流程、CUDA完整基础训练、首组正式实验日志与checkpoint检查。

尚未确认：完整模型及部分工具的官方测试反馈、Windows Tokenizer内存测试、生成脚本运行、多学习率和batch比较、消融、OWT与排行榜。不能把运行成功或理论分析自动升级为整份作业完成。

本PDF提供：细化文字答案、公式推导、预测边界、当前正式运行的两张真实学习曲线，以及代码待办。没有创建新的训练结果或伪造生成样本。

来源与进度记录

主要来源一：本地《CS336_Assignment1_2026_中文译本.md》，版本26.0.3。定位：Unicode第2节；资源核算第3、4节；生成第6节；实验第7节。

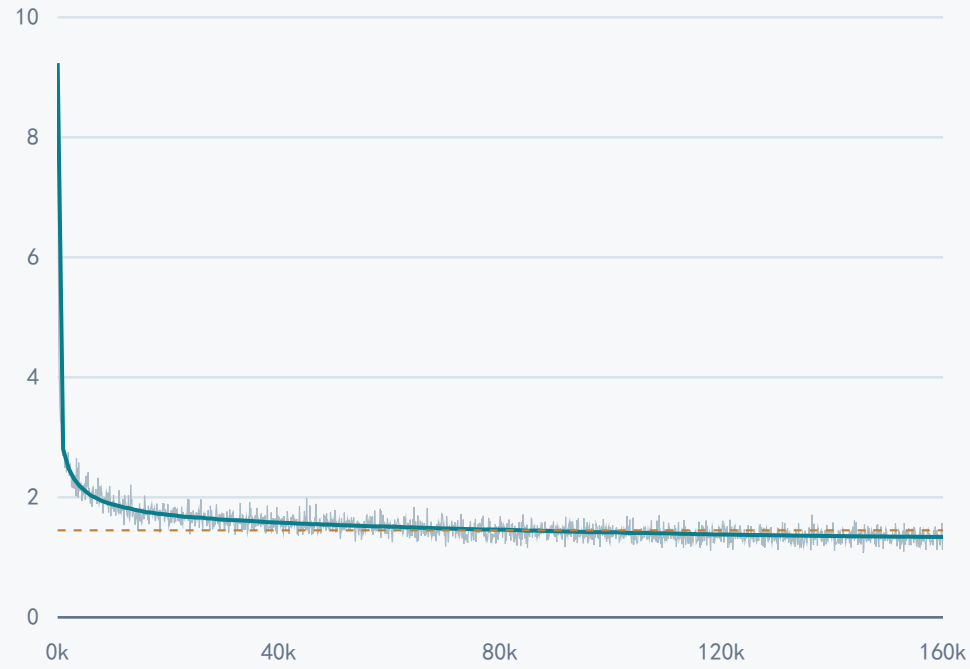
主要来源二：D:/aaa_code/assignment1-basics/runs/tinystories_lr_1e-3_b8_01中的config.json、metrics.jsonl及此前checkpoint只读检查。图表使用真实JSONL。

主要来源三：实际作业仓库的generation.py、transformer_lm.py、multihead_attention.py、tokenizer.py等已核对接口，以及你提供的编码与训练输出。课程入口：https://cs336.stanford.edu/；官方仓库：https://github.com/stanford-cs336/assignment1-basics。

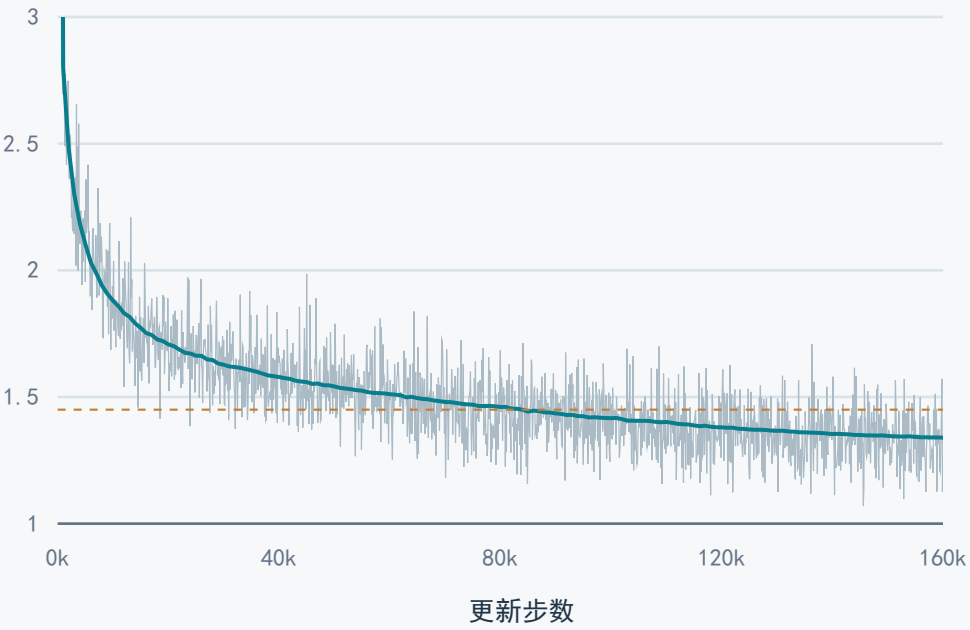
记录：已完成——详细答案与手机PDF整理；尚未确认——未运行实验及生成效果；下一步——按需阅读、审阅已有代码，不追加训练。

真实曲线：按更新步数

灰：单batch训练loss　蓝：采样验证loss　虚线：1.45



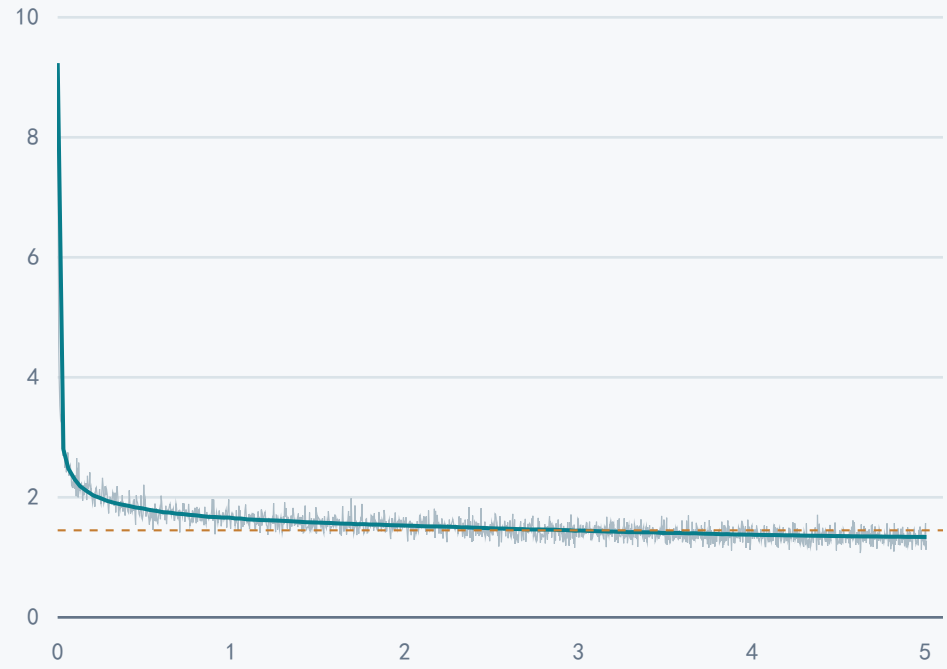
局部放大：loss 1.0 – 3.0（超出范围部分裁切）



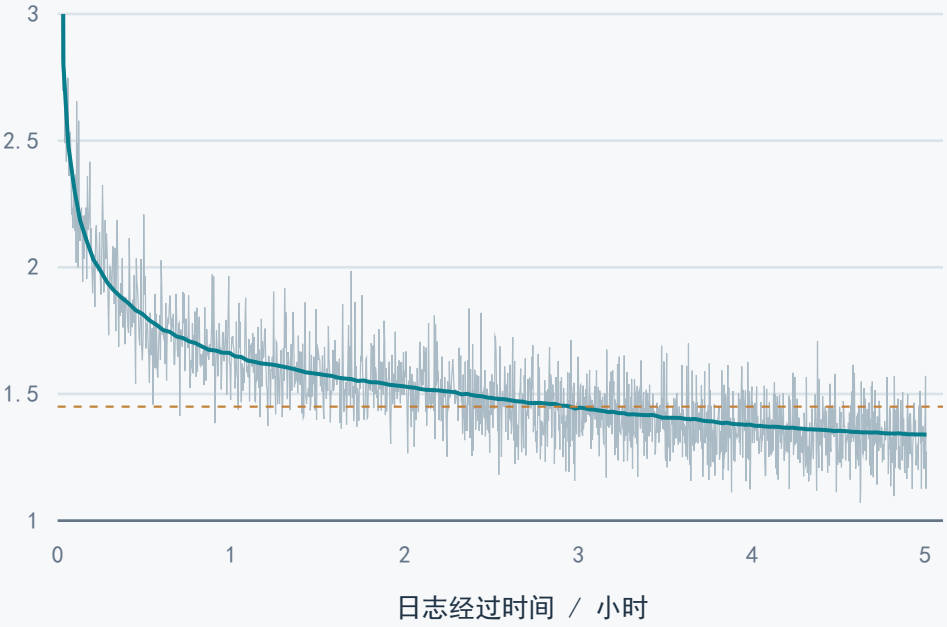
固定32个验证batch；时间不含数据扫描及模型创建。

真实曲线：按经过时间

灰：单batch训练loss　蓝：采样验证loss　虚线：1.45



局部放大：loss 1.0 – 3.0（超出范围部分裁切）



固定32个验证batch；时间不含数据扫描及模型创建。